

And the cost of all this? Zero. No licence fees, ever, and no software of doubtful origin on institute machines.

A simple migration plan

- Choose one lab, or one semester's batch, as a pilot.
- On Windows machines, install the Code::Blocks bundle that includes MinGW. On Linux machines, install gcc, g++ and Code::Blocks from the repository.
- Conduct a half-day faculty session covering how to create, compile, run and debug a program, along with the small mechanical changes to old code.
- Update the lab manual: `int main()` everywhere, standard headers only, and compilation with `-Wall` as a habit. Rework the few exercises that depend on `conio.h`, or route them through `ncurses/PDCurses`.
- Tell the students why the change is being made. The distinction

between standard C and vendor extensions will serve them for their whole careers.

- Extend the change to all labs from the following term, and retire DOSBox with honour.

The ultimate test is employability

The purpose of teaching programming is not to complete a syllabus; it is to prepare students for the world they will work in. Measured against that goal, Turbo C/C++ fails completely. Graduates who trained on it enter the industry carrying an unlearning curve, because no real-world codebase uses `<iostream.h>`; the world writes `<iostream>` and the `std::` namespace.

Making the switch gives students far more than a new compiler. Moving to GCC or Clang through Code::Blocks or VS Code introduces them to the FOSS ecosystem itself, and to the habit that marks out a good programmer everywhere: writing clean, portable, ISO-standard code that will compile anywhere, on any machine, under any operating system.

Turbo C/C++ served its purpose well; an entire generation of Indian programmers wrote their first programs on it. Respecting the past, however, does not imply not moving with the times. Our students will spend their worklife on modern, open source toolchains. Let their very first compiler be one of them. **END** 🐧

By: S. Sathyanarayanan

The author is working with Sri Sathya Sai University for Human Excellence, Gulbarga. He has more than 30 years of experience in systems management and teaching IT courses. He is an enthusiastic promoter of FOSS.

Continued from page...73

The *serial: 3* directive rolled the update out three servers at a time rather than all at once, so the site kept serving traffic from the remaining nine servers while each batch was updated and verified — a simple form of rolling deployment with no extra tooling required.

As a result:

- Release time dropped from roughly four hours to about 20 minutes of mostly unattended execution.
- Every server converged to an identical, version-controlled configuration, eliminating drift-related bugs.
- New servers were brought into production by simply adding them to the inventory and re-running the existing playbook.
- The playbook itself became living documentation — anyone could read the YAML to see exactly what a production web server looked like.

Ansible makes the automation of repetitive IT tasks like provisioning, configuring, and deploying applications an easy and agentless task. The use of standard SSH and human-readable YAML Playbooks makes its use less cumbersome. This allows teams to adopt an automation approach that is consistent, repeatable, and can be executed with reduced errors.

After verifying installation and configuring the environment variables, the next step is to start building an inventory of actual infrastructure and creating playbooks and roles. **END** 🐧

By: Dr R. Anand

The author is a professor in the Department of Computer Science and Engineering at Easwari Engineering College, Chennai, Tamil Nadu